

Informe Técnico: Optimización y Rendimiento en Arquitecturas de CPU (Bloque I)

1. Especificaciones del Sistema (Hardware Characterization)

El análisis de rendimiento se ha realizado sobre un nodo de computación de Google Colab. La caracterización del hardware es crítica para entender los límites de ancho de banda y latencia:

- **CPU:** Intel(R) Xeon(R) CPU @ 2.00GHz.
- **Topología:** 1 Socket, 1 Núcleo físico, 2 hilos lógicos (vCPUs mediante Hyper-Threading).
- **Jerarquía de Memoria Caché:**
 - **L1d/L1i:** 32 KiB (Privada por núcleo).
 - **L2:** 1 MiB (Privada).
 - **L3:** 38.5 MiB (Compartida).

2. Análisis de Optimización del Compilador (Compiler Optimization)

El uso de banderas en GCC permite cuantificar cómo la traducción del código a lenguaje máquina impacta tanto en el tiempo de ejecución como en la capacidad de cómputo. Para este test se utilizó la implementación estándar **IJK** con un tamaño de matriz **N=512**.

Resultados de Latencia y Throughput (Implementación IJK):

- **-O0 (Sin optimización):** * Tiempo medio: **2.2658 s**.
 - Rendimiento: **0.1185 GFLOPS**.
 - El procesador pasa la mayor parte del tiempo gestionando instrucciones redundantes y sufriendo por la falta de una estrategia de acceso a memoria.
- **-O2 (Optimización estándar):** * Tiempo medio: **0.4435 s**.
 - Rendimiento: **0.6053 GFLOPS**.
 - Mejora significativa respecto a -O0. El compilador optimiza el uso de registros y elimina saltos innecesarios, aunque el rendimiento sigue limitado por los fallos de caché inherentes al orden IJK.
- **-O3 (Optimización agresiva):** * Tiempo medio: **0.5029 s**.
 - Rendimiento: **0.5338 GFLOPS**.
 - **Anomalía de rendimiento:** En este caso, -O3 resulta más lento que -O2. Al intentar aplicar vectorización automática (SIMD) sobre un patrón de

acceso desordenado a memoria, el *overhead* de intentar organizar los datos para los registros vectoriales perjudica el tiempo total de ejecución.

- **-ffast-math:** * Tiempo medio: **0.4423 s**.
 - Rendimiento: **0.6069 GFLOPS**.
 - Relaja la norma IEEE 754. Al permitir simplificaciones matemáticas, logra mitigar ligeramente el retroceso visto en -O3, igualando prácticamente el rendimiento de -O2, lo que confirma que el límite real es el acceso a memoria y no la complejidad aritmética.

Riesgos de -O3 y Estabilidad:

Aunque se diseñó para reducir el tiempo medio, no siempre es ideal para cálculo científico por tres motivos:

1. **Inestabilidad Numérica:** Al reordenar operaciones para vectorizar, pueden variar los resultados debido a que la aritmética de punto flotante no es asociativa (el orden de los factores sí altera el producto final por redondeos).
2. **Tamaño del Binario:** El *loop unrolling* excesivo puede aumentar el tamaño del código hasta saturar la Caché de Instrucciones (L1i), lo que en arquitecturas limitadas incrementa el tiempo de ejecución.
3. **Dificultad de Depuración:** La optimización elimina la correspondencia directa entre el código fuente y el binario, imposibilitando seguir el flujo lógico del programa en caso de error.

3. Localidad de Datos e Intercambio de Bucles

En esta sección se demuestra que el orden de los bucles es un factor más determinante que la potencia bruta de la CPU. Se comparó la multiplicación de matrices estándar (**IJK**) frente a la versión optimizada (**IKJ**) utilizando el nivel de optimización máximo del compilador (**-O3**).

Comparativa de Rendimiento (N=512, Opt. -O3)

Algoritmo	Orden de Bucles	Tiempo Medio	Rendimiento (GFLOPS)
Básico (Referencia)	IJK	0.5029 s	0.5338 GFLOPS
Optimizado	IKJ	0.0549 s	4.8863 GFLOPS

Análisis del resultado:

El algoritmo **IKJ** es **9.15 veces más rápido** que el IJK. Esta mejora masiva ocurre sin cambiar ni una sola operación aritmética, actuando únicamente sobre la forma en que el programa accede a la memoria.

Justificación Técnica: El impacto en las Líneas de Caché

La diferencia de rendimiento se justifica por la eficiencia en el uso de la jerarquía de memoria del Intel Xeon:

1. **Fallas de Caché en IJK:** En el orden IJK, el bucle interno recorre la matriz B por columnas. Al ser C++ un lenguaje *Row-Major*, la CPU se ve obligada a saltar entre filas para obtener cada dato. Esto provoca constantes **Cache Misses**, ya que cada salto cae fuera de la **Línea de Caché de 64 bytes** que el procesador acaba de cargar. El procesador desperdicia el 90% del tiempo esperando a la RAM.
2. **Localidad Espacial en IKJ:** Al intercambiar los bucles a IKJ, el acceso a las matrices B y C en el bucle más interno es contiguo.
 - a. **Aprovechamiento de la Línea de Caché:** Al solicitar un dato `double`, el hardware carga automáticamente los 7 siguientes que están en la misma línea. Al estar el código recorriendo la fila en orden, esos 7 datos ya están en la **Caché L1** cuando se necesitan.
 - b. **Efectividad de SIMD:** Tus datos muestran que con IKJ el rendimiento sube a **4.88 GFLOPS**. Esto confirma que el compilador pudo aplicar vectorización **AVX2**, procesando múltiples datos por ciclo de reloj, algo que era imposible con los datos dispersos del orden IJK.

4. Escalabilidad con OpenMP (Parallel Computing)

En esta fase se evalúa la capacidad de aceleración mediante el uso de hilos de ejecución paralela con OpenMP, utilizando la directiva `#pragma omp parallel for`.

Cálculo del Speedup

El **Speedup (S)** representa la ganancia de velocidad al pasar de un hilo a varios. Tomando los resultados coherentes obtenidos para una matriz de $N=1024$:

- **Rendimiento Multihilo (2 hilos): 4.2574 GFLOPS.**
- **Tiempo Paralelo (T_p): 0.5044 s.**
- **Tiempo Secuencial Estimado (T_s): 0.8776 s** (Basado en la ejecución optimizada monohilo).

- **Speedup:** $S=0.8776/0.5044=1.74x$.

Análisis de escalabilidad: El factor de aceleración de **1.74x** es sólido para el hardware de Colab. No alcanza el 2x teórico debido a que los dos hilos lógicos comparten un único núcleo físico (**Hyper-Threading**). Esto genera competencia por las unidades funcionales y la caché, además del *overhead* de sincronización de hilos.

SAXPY: ¿Escalabilidad Lineal o Límite de Memoria?

A diferencia de la multiplicación de matrices (que es un problema de cálculo intenso), los resultados en el benchmark SAXPY muestran una escalabilidad casi nula.

- Diagnóstico (Memory Bound): SAXPY tiene una intensidad aritmética muy baja (pocas operaciones por cada dato leído). El procesador es mucho más rápido que la memoria RAM.
- Discusión: SAXPY no escala linealmente porque está limitado por el ancho de banda de la RAM. Al saturarse el bus de datos, añadir más hilos no mejora el rendimiento; los núcleos pasan la mayor parte del tiempo esperando a que los datos lleguen desde la memoria principal. Este fenómeno confirma la existencia del "Muro de la Memoria".

5. Conclusiones Técnicas

Tras el análisis de rendimiento en el hardware de Google Colab, se extraen las siguientes conclusiones clave:

1. **Técnica de Mayor Impacto (Localidad de Datos):** La optimización más crítica fue el intercambio de bucles (**IKJ**), que elevó el rendimiento de **0.53 a 4.88 GFLOPS (mejora del 915%)**. Esto confirma que el orden de acceso a memoria es más determinante que el número de hilos.
2. **Sinergia con el Compilador:** Las banderas de optimización agresiva (**-O3**) solo son efectivas cuando hay localidad de datos. En el orden IJK resultaron contraproducentes, mientras que en **IKJ** habilitaron el máximo potencial de la **vectorización SIMD (AVX2)**.
3. **Eficiencia del Paralelismo:** Se logró un **Speedup de 1.74x** con OpenMP. La saturación antes del 2x teórico se debe a la arquitectura de **Hyper-Threading**, donde los hilos compiten por los recursos de un único núcleo físico.
4. **Cuellos de Botella (Compute vs. Memory Bound):** * La **Multiplicación de Matrices** escaló con éxito al ser limitada por cómputo (*Compute Bound*).
 - a. **SAXPY** se estancó debido al ancho de banda de la RAM ("**Muro de la Memoria**"), confirmando su naturaleza *Memory Bound*.

Resumen Final:

En este hardware, la **jerarquía de memoria** es el factor limitante principal. La optimización del acceso a las líneas de caché es la base obligatoria antes de aplicar cualquier estrategia de paralelismo.